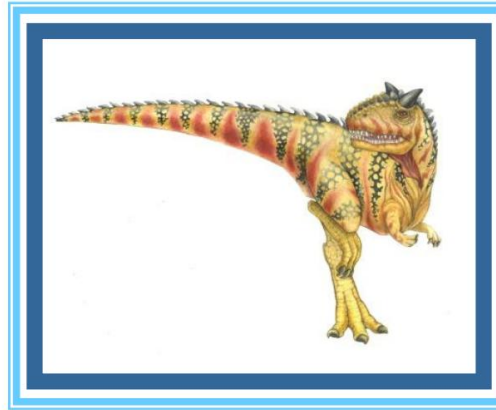


الفصل الخامس: العملية المزامنة

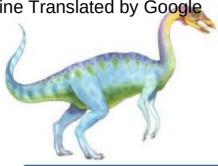




الفصل الخامس: مزامنة العمليات

- خلفية
- مشكلة القسم الحرج
- حل بيترسون
- أجهزة المزامنة
- أقفال مونتيس
- إشارات المرور
- المشاكل الكلاسيكية للمزامنة
- الشاشات
- أمثلة المزامنة
- النهج البديلة

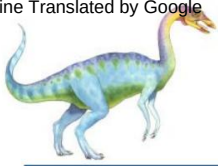




أهداف

- لتقديم مفهوم مزامنة العملية.
- لتقديم مشكلة القسم الحرج، والتي يمكن استخدامها لحلها لضمان اتساق البيانات المشتركة
- لتقديم حلول برمجية وأجهزة لمشكلة القسم الحرج
- لفحص العديد من مشاكل مزامنة العمليات الكلاسيكية
- لاستكشاف العديد من الأدوات المستخدمة لحل مشاكل مزامنة العمليات

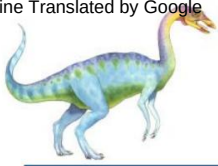




خلفية

- يمكن تنفيذ العمليات في وقت واحد
- قد يتم مقاطعته في أي وقت، مما يؤدي إلى إكمال التنفيذ جزئيًا
- قد يؤدي الوصول المتزامن إلى البيانات المشتركة إلى عدم تناسق البيانات
- يتطلب الحفاظ على اتساق البيانات آليات لضمان التنفيذ المنظم للعمليات المتعاونة
- توضيح للمشكلة:
 لنفترض أننا نريد توفير حل لمشكلة المستهلك-المنتج الذي يملأ المخازن المؤقتة. يمكننا تحقيق ذلك باستخدام عداد صحيح ويتخلف عنه المخازن المؤقتة للمستهلك. في البداية، يكون العداد مضبوطًا على 0. يزيده المنتج بعد إنتاج مخزن مؤقت جديد،





منتج

بينما (صحيح) {

/إنتاج عنصر في المنتج التالي /

بينما (العداد ؛ BUFFER_SIZE ==

/* لا تفعل شيئاً */

المخزن المؤقت [في] = المنتج التالي؛

في = (في % (1 + حجم المخزن المؤقت؛

عداد+؛+

}

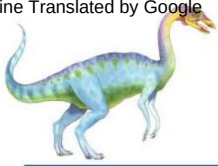




مستهلك

```
بينما (صحيح) {  
    بينما (العداد == 0)  
        /* لا تفعل شيئاً */  
        = buffer[out]; out = (out + 1) % BUFFER_SIZE;  
        next_consumed  
        عداد--;  
        /* استهلك العنصر في الاستهلاك التالي */  
}
```





حالة السباق

يمكن تنفيذ `counter++`

السجل = العداد
 السجل = السجل + 1
 العداد = السجل + 1

يمكن تنفيذ العداد-- كـ

السجل = العداد
 السجل = السجل - 1
 العداد = السجل - 1

خذ بعين الاعتبار هذا التنفيذ المتداخل مع "count = 5" في البداية:

S0: المنتج ينفذ السجل = العداد {السجل = 5} = 1
 S1: المنتج ينفذ السجل = السجل + 1 {السجل = 6} = 1
 S2: المستهلك ينفذ السجل = العداد {السجل = 5} = 2
 S3: يقوم المستهلك بتنفيذ السجل = السجل - 1 {السجل = 4} = 2
 S4: المنتج ينفذ العداد = السجل {العداد = 6} = 1
 S5: عداد تنفيذ المستهلك = السجل {العداد = 4} = 2





مشكلة القسم الحرج

- نضع في اعتبارك نظام $pn-1 \dots$ للعمليات {
- تحتوي كل عملية على جزء **حرج** من الكود
- قد تتضمن العملية تغيير المتغيرات الشائعة، أو تحديث الجدول، أو كتابة ملف، وما إلى ذلك
- عندما تكون عملية واحدة في قسم حرج، لا يجوز أن تكون أي عملية أخرى في قسمها الحرج

- **مشكلة القسم الحرج وحل هذه المشكلة**
- يجب على كل عملية أن تطلب الإذن لدخول القسم الحرج في **قسم الدخول**، ويمكن أن يتبع القسم الحرج **بقسم الخروج**، ثم **قسم الباقي**





القسم الحرج



الهيكل العام للعملية

```
do {
```

```
    entry section
```

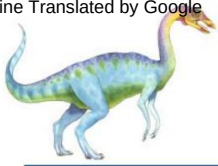
```
    critical section
```

```
    exit section
```

```
    remainder section
```

```
} while (true);
```





خوارزمية للعملية

{ يفعل

بينما (turn == j)

القسم الحرج

دورة = ج؛

قسم الباقي

{بينما (صحيح)؛





حل مسألة القسم الحرج

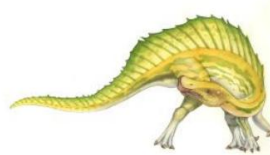
1. **الاستبعاد المتبادل** - إذا كانت العملية تُنفذ في قسمها الحرج **بلا** يمكن لأي عمليات أخرى أن تُنفذ في أقسامها الحرجة

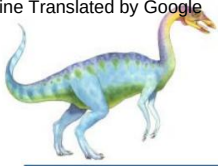
2. **التقدم** - إذا لم يتم تنفيذ أي عملية في قسمها الحرج وكانت هناك بعض العمليات التي ترغب في الدخول إلى قسمها الحرج ، فلا يمكن تأجيل اختيار العمليات التي ستدخل القسم الحرج بعد ذلك إلى أجل غير مسمى

3. **الانتظار المحدود** - يجب أن يوجد حد لعدد المرات التي يُسمح فيها للعمليات الأخرى بالدخول إلى أقسامها الحرجة بعد أن تقدم العملية طلبًا للدخول إلى قسمها الحرج وقبل الموافقة على هذا الطلب

□ افترض أن كل عملية يتم تنفيذها بسرعة غير صفيرية

ن □ لا يوجد افتراض بشأن **السرعة النسبية** للعمليات





التعامل مع القسم الحرج في نظام التشغيل

طريقتان تعتمدان على ما إذا كانت النواة استباقية أم غير استباقية

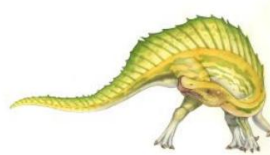


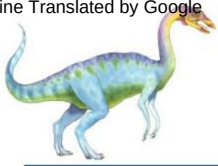
استباقي - يسمح بمنع العملية من العمل عند التشغيل في وضع النواة



غير استباقي - يتم تشغيله حتى الخروج من وضع النواة، أو الحظر، أو الاستسلام الطوعي لوحدة المعالجة المركزية

ةيلاخ بشكل أساسي من شروط السباق في وضع النواة





حل بيترسون



وصف خوارزمي جيد لحل المشكلة



حل عمليتين



افترض أن تعليمات لغة الآلة الخاصة بالتحميل والتخزين ذرية ؛ أي لا يمكن مقاطعتها



تتقاسم العمليتان متغيرين:



بدوره؛



العلم البوليانى[2]

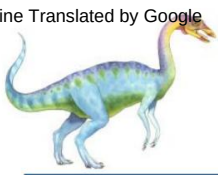


يشير الدور المتغير إلى من هو الدور الذي يجب أن يدخل القسم الحرج



يتم استخدام مجموعة الأعلام للإشارة إلى ما إذا كانت العملية جاهزة لدخول القسم الحرج. $\text{flag}[i] =$ يعني أن العملية P_i جاهزة !
حقيقي





خوارزمية لعملية باي

{ يفعل

العلم[i] = صحيح؛

دورة = ج؛

بينما (flag[j] && turn == j)؛

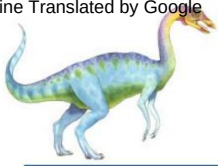
القسم الحرج

العلم[i] = خطأ؛

قسم الباقي

{بينما (صحيح)؛





حل بيترسون (تابع)



يمكن إثبات استيفاء متطلبات CS الثلاثة:

1. يتم الحفاظ على الاستبعاد المتبادل

يدخل P_i إلى CS فقط إذا:

إما $turn = i$ أو $flag[j] = false$

2. تم استيفاء متطلبات التقدم

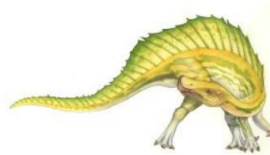
3. تم استيفاء متطلب الانتظار المحدود





أجهزة المزامنة

- توفر العديد من الأنظمة دعمًا للأجهزة لتنفيذ كود القسم الحرج.
- جميع الحلول أدناه تعتمد على فكرة **القفل**
- حماية المناطق الحرجة عبر الأقفال
- المعالجات الأحادية - قد تؤدي إلى تعطيل المقاطعات
- سيتم تنفيذ الكود الجاري تشغيله حاليًا دون أي استثناء
- غير فعال بشكل عام على أنظمة المعالجات المتعددة
- أنظمة التشغيل التي تستخدم هذا غير قابلة للتوسع على نطاق واسع
- توفر الآلات الحديثة تعليمات خاصة للأجهزة الذرية
- **ذري** = غير قابل للانقطاع
- إما اختبار كلمة الذاكرة أو تعيين القيمة
- أو قم بتبادل محتويات كلمتين من الذاكرة





حل مشكلة القسم الحرج باستخدام الأقفال

{ يفعل }

الحصول على القفل

القسم الحرج

قفل التحرير

قسم الباقي

{ بينما (صحيح)؛ }





تعليمات الاختبار والتعيين

تعريف:

اختبار وتعيين منطقي (منطقي * الهدف)

{

بولياني $rv = \text{الهدف}^*$ ؛

$\text{الهدف}^* = \text{صحيح}$ ؛

عودة: rv

}

1. يتم التنفيذ ذريًا. 2. إرجاع القيمة الأصلية للمعلمة المرسلة. 3. تعيين القيمة

الجديدة للمعلمة المرسلة إلى "TRUE".





الحل باستخدام test_and_set()



قفل متغير بولياني مشترك، تم تهيئته إلى FALSE



حل:

{ يفعل

بينما (test_and_set(&lock))

/* لا تفعل شيئاً */

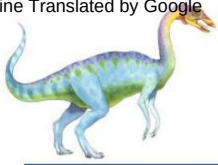
/* قسم حرج */

قفل = خطأ؛

/* قسم الباقي */

{بينما (صحيح)؛





تعليمات المقارنة والمبادلة

تعريف:

```
int قارن (int *_and_swap القيمة، int المتوقعة، int القيمة الجديدة) {
    int temp = *القيمة;
```

```
    إذا (*القيمة == المتوقعة)
    *القيمة = القيمة الجديدة؛
    درجة الحرارة العائدة؛
}
```

1. تم التنفيذ ذريًا

2. إرجاع القيمة الأصلية للمعلمة المرسلة "القيمة"

3. اضبط المتغير "value" على قيمة المعلمة "new_value" المُمررة، ولكن بشرط أن تكون "value" == "expected" أي أن عملية التبديل تتم فقط في هذا الشرط.





الحل باستخدام compare_and_swap



تم تهيئة "قفل" عدد صحيح مشترك إلى 0؛



حل:

{ يفعل

بينما /* (compare_and_swap(&lock, 0, 1) != 0) لا تفعل شيئاً */

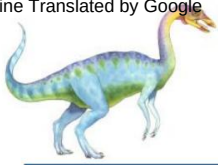
/* قسم حرج */

القفل = 0؛

/* قسم الباقي */

{بينما (صحيح)؛





الاستبعاد المتبادل المحدود بالانتظار مع test_and_set

```

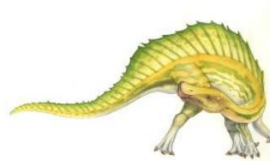
{
    يفعل
    انتظار[i] = صحيح؛
    المفتاح = صحيح؛
    أثناء (الانتظار[i] والمفتاح)

    المفتاح = test_and_set(&lock);
    انتظار[i] = خطأ؛

    /* قسم حرج */
    ج = (ي % (1 + ن)؛
    بينما ((j != i) && !waiting[j]))
    ج = (ج % (1 + ن)؛
    إذا (ج == i)
    قفل = خطأ؛
    آخر

    انتظار[ج] = خطأ؛
    /* قسم الباقي */
    {بينما (صحيح)؛

```





أقفال موتكس

- الحلول السابقة معقدة وغير متاحة عمومًا لمبرمجي التطبيقات
- يقوم مصممو أنظمة التشغيل ببناء أدوات برمجية لحل مشكلة القسم الحرج
- أبسطها هو قفل الميكس
- حماية قسم حرج من خلال الحصول على قفل أولاً ثم تحرير القفل
- متغير منطقي يشير إلى ما إذا كان القفل متاحًا أم لا
- يجب أن تكون المكالمات إلى `acquire()` و `release()` ذرية
- يتم تنفيذها عادة من خلال تعليمات ذرية للأجهزة
- لكن هذا الحل يتطلب الانتظار الطويل
- لذلك يسمى هذا القفل **قفل الدوران**





الاستحواذ () والإفراج ()

□

يكتسب () {
بينما (!) متاح

/*; انتظار مشغول */

متاح = خطأ؛

}

□

يطلق () {

متاح = صحيح؛

}

□

يفعل {

يكتسب

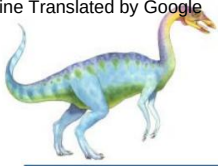
القسم الحرج

يطلق

قسم الباقي

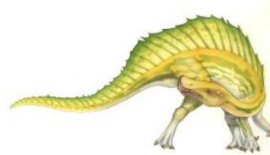
{بينما (صحيح)؛





إشارة ضوئية

- أداة مزامنة توفر طرقًا أكثر تطورًا (من أقفال Mutex) للعملية لمزامنة أنشطتها.
- إشارة ضوئية - متغير عدد صحيح
- لا يمكن الوصول إليها إلا من خلال عمليتين غير قابلتين للتجزئة (ذريتين)
- wait() و signal()
- كان يسمى في الأصل P() و V()
- تعريف عملية wait()
 - انتظر(س) {
 - بينما (S <= 0)
 - // مشغول الانتظار
 - س--;
 - }
- تعريف عملية الإشارة ()
 - إشارة(س) {
 - س++;
 - }





استخدام إشارات المرور



عد الإشارة - يمكن أن تتراوح قيمة العدد الصحيح عبر نطاق غير مقيد



إشارة ثنائية - يمكن أن تتراوح القيمة الصحيحة بين 0 و 1 فقط



نفس قفل الميكس



يمكنه حل مشاكل المزامنة المختلفة



S2 أن يحدث قبل ~~الوقت المطلوب~~

إنشاء إشارة "مزامنة" مبدئية إلى 0

س1:

س1؛

إشارة (مزامنة)؛

ص2:

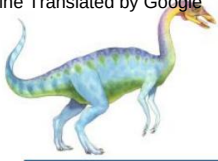
الانتظار (المزامنة)؛

S2;



يمكن تنفيذ إشارة العد





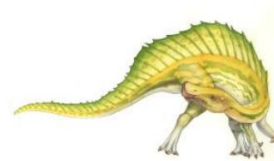
تنفيذ إشارات المرور

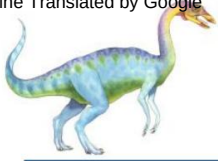
- يجب ضمان عدم تمكن عمليتين من تنفيذ `wait()` و `signal()` على نفس الإشارة في نفس الوقت
- وبالتالي، يصبح التنفيذ هو مشكلة القسم الحرج حيث يتم وضع رمز الانتظار والإشارة في القسم الحرج
- قد يكون الآن **مشغولاً بالانتظار** في تنفيذ القسم الحرج

□ لكن كود التنفيذ قصير

□ الانتظار قليلاً إذا كان القسم الحرج نادرًا ما يكون مشغولاً

- لاحظ أن التطبيقات قد تستغرق وقتًا طويلاً في الأقسام الحرجة وبالتالي هذا ليس حلاً جيداً





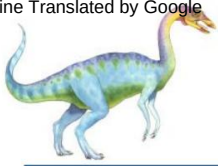
تنفيذ إشارة الإشارة بدون انتظار مشغول

- مع كل إشارة ضوئية هناك قائمة انتظار مرتبطة بها
- يحتوي كل إدخال في قائمة الانتظار على عنصرين من البيانات:
 - القيمة (من نوع عدد صحيح)
 - مؤشر إلى السجل التالي في القائمة
- عمليتان:
 - كتلة - وضع العملية التي تستدعي العملية في قائمة الانتظار المناسبة
 - الاستيقاظ - إزالة إحدى العمليات الموجودة في قائمة الانتظار ووضعها في قائمة الاستعداد
- ```

typedef{
 هيكل
 قيمة! int
 عملية هيكل *قائمة؛
 إشارة ضوئية؛

```





# التنفيذ بدون انتظار مشغول (تابع)

انتظر (إشارة الإشارة) { \*S)

س- < القيمة --؛

إذا S- < القيمة { 0 <

أضف هذه العملية إلى S->list؛

حاجز()؛

}

}

إشارة (إشارة ضوئية) { \*S)

س- < القيمة ++؛

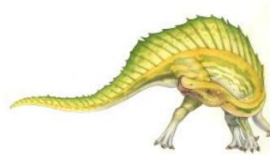
إذا S- < القيمة { 0 <=

إزالة عملية P من القائمة S؛

الاستيقاظ (P)؛

}

}





# الجمود والمجاعة

- الجمود - عمليتان أو أكثر تنتظران إلى أجل غير مسمى حدثًا لا يمكن أن يحدث إلا بواسطة عملية واحدة من العمليات المنتظرة

- يكون هناك إشارتان ضوئيتان تم تهيئتهما إلى 1

| ص           | ص           |
|-------------|-------------|
| انتظر(س)؛   | انتظر(س)؛   |
| انتظر(س)؛   | انتظر(س)؛   |
| ...         | ...         |
| الإشارة(S)؛ | الإشارة(S)؛ |
| الإشارة(S)؛ | الإشارة(S)؛ |

- التجوية - حظر غير محدد

- لا يمكن إزالة العملية أبدًا من قائمة انتظار الإشارات التي تم تعليقها فيها

- عكس الأولوية - مشكلة الجدولة عندما تحتفظ العملية ذات الأولوية الأقل بالقفل المطلوب من قبل العملية ذات الأولوية الأعلى

- تم حلها عبر بروتوكول الأولوية والوراثة





# المشاكل الكلاسيكية للمزامنة



المشاكل الكلاسيكية المستخدمة لاختبار مخططات المزامنة المقترحة حديثاً



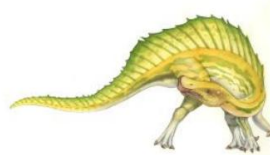
مشكلة المخزن المؤقت المحدود



مشكلة القراء والكتاب



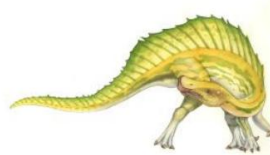
مشكلة الفلاسفة في تناول الطعام





# مشكلة المخزن المؤقت المحدود

- ن      المخازن المؤقتة، كل منها يمكن أن تحتوي على عنصر واحد
- تم تهيئة متزامن الإشارة الضوئية إلى القيمة 1
- تم تهيئة إشارة المرور بالكامل إلى القيمة 0
- تم تهيئته إشارة فارغة إلى القيمة  $n$





# مشكلة المخزن المؤقت المحدود (تابع)



هيكل عملية الإنتاج

{ يفعل

...

\*/إنتاج عنصر في next\_produced \*

...

انتظر(فارغ)؛

انتظر(xetum)؛

...

\*/أضف المنتج التالي إلى المخزن المؤقت \*/

...

إشارة (متزامنة)؛

إشارة (كاملة)؛

{بينما (صحيح)؛





# مشكلة المخزن المؤقت المحدود (تابع)



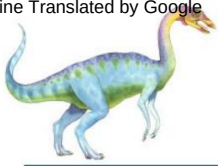
هيكل عملية المستهلك

```

 { يفعل
 انتظر(كامل)؛
 انتظر(xetum);
 ...
 /*إزالة عنصر من المخزن المؤقت إلى */ next_consumed
 ...
 إشارة (متزامنة)؛
 إشارة(فارغة)؛
 ...
 /*استهلك العنصر في الاستهلاك التالي */
 ...
 }بينما (صحيح)؛

```





# مشكلة القراء والكتاب

- تتم مشاركة مجموعة البيانات بين عدد من العمليات المتزامنة
- ~~إجراء أي-تقريباً~~ فقط مجموعة البيانات؛ فهم يفعلون ذلك
- الكتاب -يمكنهم القراءة والكتابة
- المشكلة -السماح لعدة قراء بالقراءة في نفس الوقت
- يمكن لكاتب واحد فقط الوصول إلى البيانات المشتركة في نفس الوقت
- هناك عدة اختلافات في كيفية النظر إلى القراء والكتاب -وكلها تنطوي على شكل من أشكال الأولويات
- البيانات المشتركة
  - مجموعة البيانات
  - تم تهيئة Semaphore rw\_mutex إلى 1
  - تم تهيئة متزامن الإشارة الضوئية إلى 1
  - تم تهيئة عدد صحيح read\_count إلى 0





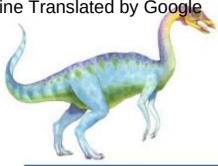
# مشكلة القراءة والكتاب (تابع)



هيكل عملية الكاتب

```
{ يفعل
انتظر(xetum_wr);
...
/* تم تنفيذ الكتابة */
...
إشارة(xetum_wr);
{بينما (صحيح);
```





# مشكلة القراء والكتاب (تابع)



## بنية عملية القراءة

{ يفعل

انتظر(xetum);

قراءة\_العدد++;

إذا (read\_count == 1) انتظر

(mutex); إشارة (rw\_mutex);

...

/\*تم إجراء القراءة\*/

...

انتظر(xetum);

قراءة العد--;

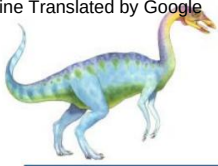
إذا (read\_count == 0) إشارة

(rw\_mutex);

إشارة (متزامنة);

{بينما (صحيح);





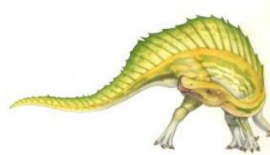
# اختلافات مشكلة القراء والكتاب

□ أولاً التنوع -لا يظل القارئ منتظرًا ما لم يكن لدى الكاتب إذن لاستخدام الكائن المشترك

□ ثانية التنوع -بمجرد أن يصبح الكاتب جاهزًا، فإنه يقوم بالكتابة في أسرع وقت ممكن

□ قد يكون كلاهما يعاني من المجاعة مما يؤدي إلى المزيد من الاختلافات

□ تم حل المشكلة في بعض الأنظمة عن طريق توفير أقفال القراءة والكتابة للنواة





# مشكلة الفلاسفة في تناول الطعام



- يقضي الفلاسفة حياتهم بالتناوب بين التفكير والأكل
- لا تتفاعل مع جيرانك، حاول من حين لآخر التقاط عيدان تناول الطعام (واحدًا في كل مرة) لتناول الطعام من الوعاء
- نحتاج إلى كليهما لتناول الطعام، ثم إطلاق كليهما عند الانتهاء
- في حالة 5 فلاسفة
- البيانات المشتركة
- وعاء من الأرز (مجموعة البيانات)
- تم تهيئته عصا الإشارة [5] إلى 1





# خوارزمية مشكلة تناول الطعام - الفلاسفة

بنية الفيلسوف {

أنا

□

انتظر (عود تناول الطعام[i]);

انتظر (chopStick[ (i + 1) % 5 ] );

// يأكل

إشارة (عصا تناول الطعام[i]);

الإشارة (عود تناول الطعام] i)؛ (5] % 1) +

// يفكر

{بينما (صحيح);

□

ما هي المشكلة مع هذه الخوارزمية؟





# خوارزمية مشكلة تناول الطعام - الفلاسفة (تابع)



معالجة الجمود



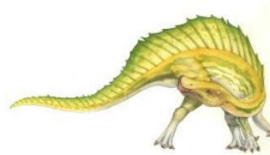
اسمح لأربعة فلاسفة كحد أقصى بالجلوس على الطاولة في نفس الوقت.

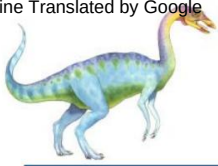


اسمح للفيلسوف أن يلتقط الشوك فقط إذا كان كلاهما متاحًا (يجب أن يتم التقاط الشوك في قسم حرج).



استخدم حلاً غير متماثل -فيلسوف ذو رقم فردي يلتقط أولاً عود الطعام الأيسر ثم عود الطعام الأيمن. فيلسوف ذو رقم زوجي يلتقط أولاً عود الطعام الأيمن ثم عود الطعام الأيسر.





# مشاكل مع إشارات المرور

- الاستخدام غير الصحيح لعمليات الإشارة:
- إشارة (مزامنة) .... انتظار (مزامنة)
- انتظر (متزامن) ... انتظر (متزامن)
- حذف الانتظار (mutex) أو الإشارة (mutex) (أو كليهما)
- من الممكن حدوث طريق مسدود ومجاعة.





# الشاشات

- تجريد عالي المستوى يوفر آلية ملائمة وفعالة لمزامنة العمليات
- المتغيرات الداخلية يمكن الوصول إليها فقط من خلال الكود الموجود داخل نوع البيانات المجردة إجراء
- لا يجوز أن تكون هناك سوى عملية واحدة نشطة داخل الشاشة في وقت واحد
- ولكنها ليست قوية بما يكفي لنمذجة بعض مخططات المزامنة

اسم الشاشة

{

//إعلانات المتغيرات المشتركة

الإجراء { .... } (... ) P1

الإجراء {.....} (... ) Pn

كود التهيئة { ... } (... )

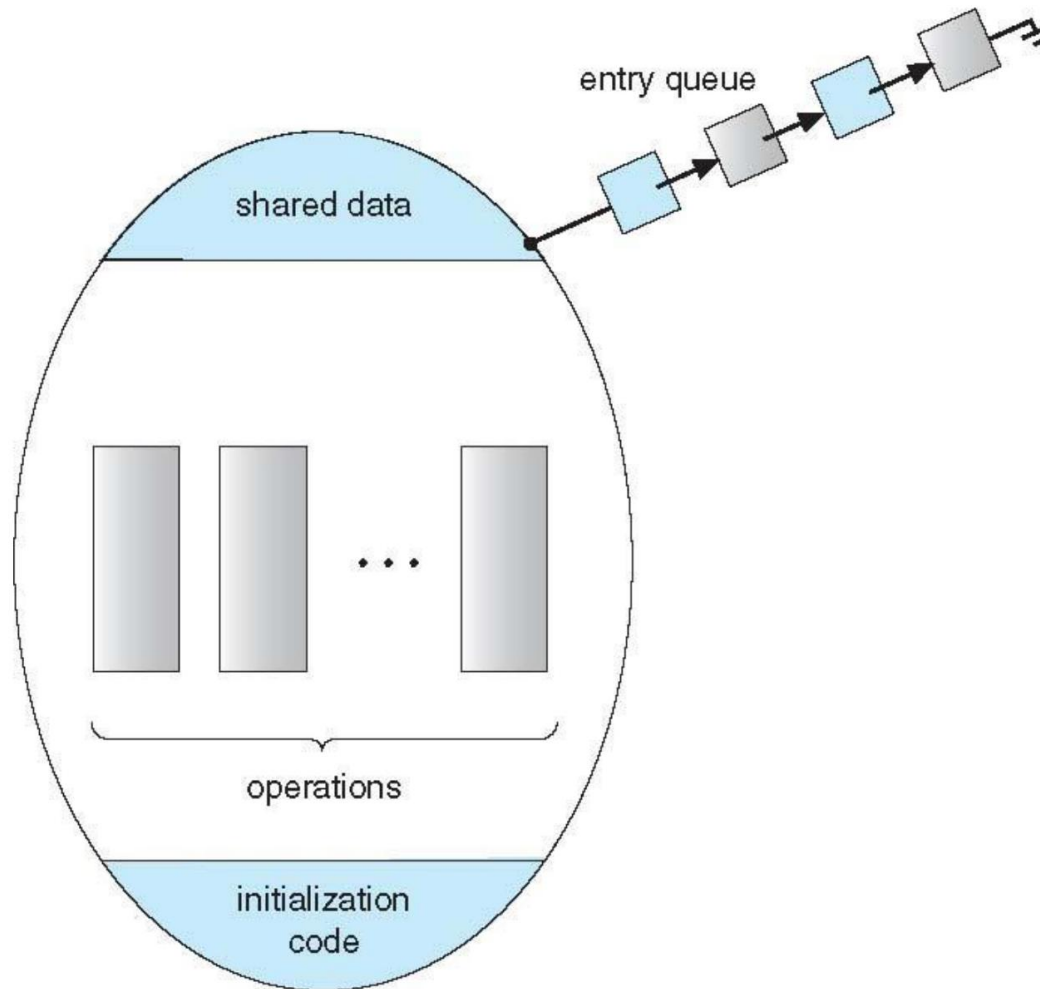
}

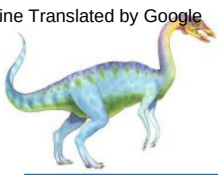
}





# منظر تخطيطي للشاشة





# متغيرات الحالة



الحالة  $x, y$ !



يُسمح بإجراء عمليتين على متغير الشرط:



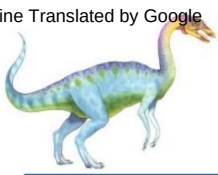
-  $x.wait()$  يتم تعليق العملية التي تستدعي العملية حتى  $x.signal()$



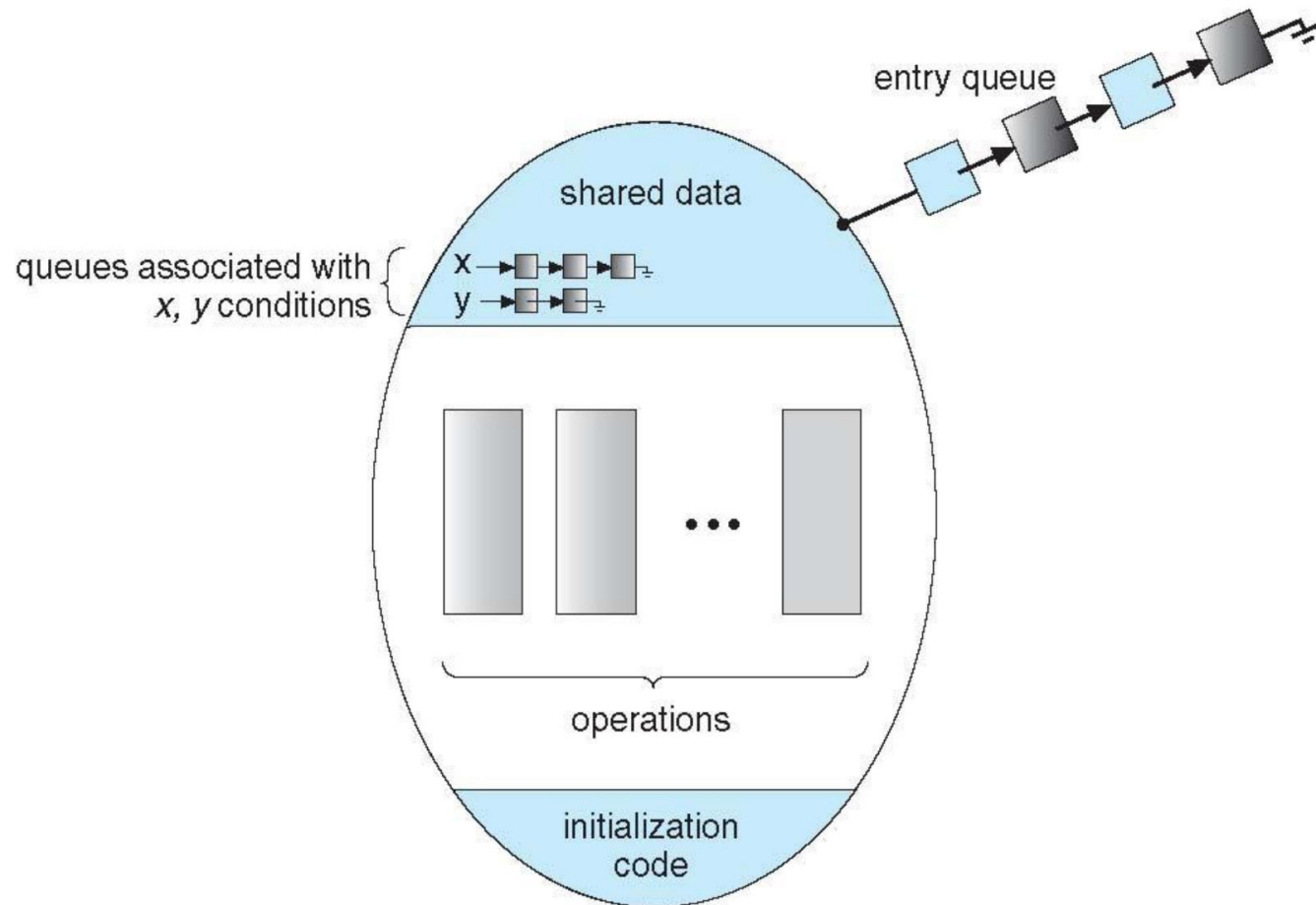
-  $x.signal()$  يستأنف إحدى العمليات (إن وجدت) التي  
تم استدعاء  $x.wait()$

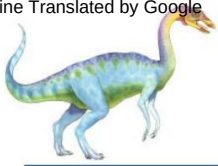
□ إذا لم يكن هناك  $x.wait()$  على المتغير، فلن يكون له أي تأثير على المتغير





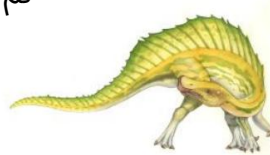
# مراقبة مع متغيرات الحالة

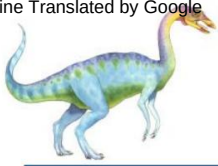




# خيارات متغيرات الحالة

- إذا استدعت العملية  $P$   $x.signal()$  وتم تعليق العملية  $Q$  في  $x.wait()$  فماذا يجب أن يحدث بعد ذلك؟
- لا يمكن تنفيذ كل من  $P$  و  $Q$  بالتوازي. إذا استؤنف  $Q$ ، فيجب على  $P$  الانتظار.
- تشمل الخيارات
  - الإشارة والانتظار - ينتظر  $P$  حتى يغادر  $Q$  الشاشة أو ينتظر حالة أخرى
  - الإشارة والاستمرار - ينتظر  $Q$  حتى يغادر  $P$  الشاشة أو ينتظر حالة أخرى
  - كلاهما له إيجابيات وسلبيات - يمكن لمنفذ اللغة أن يقرر
  - الشاشات التي تم تنفيذها في تسوية باسكال المتزامنة
  - $P$  تنفيذ الإشارة يترك الشاشة على الفور،  $Q$  هو استأنفت
  - تم تنفيذه بلغات أخرى بما في ذلك Mesa وC# وJava





# مراقبة الحلول لفلاسفة الطعام

## DiningPhilosophers مراقبة

{

حالة {التفكير، الجوع، الأكل}: [5]

حالة الذات: [5]

التقاط الفراغ { (int i)

الحالة [i] = جائع؛

اختبار(i);

إذا (state[i] != EATING) self[i].wait;

}

void putdown (int i) { state[i] = THINKING;

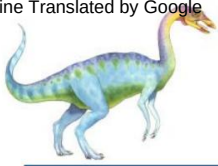
// اختبار الجيران الأيمن والأيسر

اختبار((i + 4) % 5);

اختبار((i + 1) % 5);

}





# حل لفلاسفة الطعام (تابع)

```

 اختبار باطل { (int i)
 إذا ((state[(i + 4) % 5] != EATING) &&
 state[i] == جائع) &&
 (حالة [(i + 1) % 5] != الأكل) (
 الحالة[i] = تناول الطعام؛
 self[i].signal ();
 }
 }

```

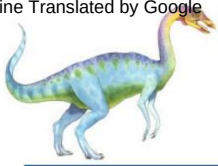
رمز التهيئة () { ل (int i = 0؛ i < 5؛ i++)

```

 الحالة[i] = التفكير؛
 }
}

```





# حل لفلاسفة الطعام (تابع)



يستدعي كل فيلسوف عمليتي `pickup()` و `putdown()` بالتسلسل التالي:

```
DiningPhilosophers.pickup(i);
```

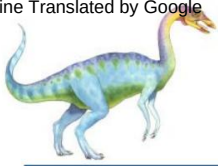
يأكل

```
DiningPhilosophers.putdown(i);
```



لا يوجد طريق مسدود، ولكن المجاعة ممكنة





# مراقبة التنفيذ باستخدام الإشارات الضوئية



المتغيرات

إشارة ضوئية متزامنة؛ // (في البداية 1) =  
 الإشارة التالية؛ // (في البداية 0) =  
 int next\_count = 0؛



فلا يتم إجراء استبداله بـ

انتظر (xetum)؛

...

جسم F؛

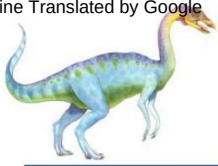
...

إذا (العدد التالي 0) >  
 الإشارة (التالية)  
 وإلا إشارة؛ (mutex)



يتم ضمان الاستبعاد المتبادل داخل الشاشة





# مراقبة التنفيذ - متغيرات الشرط



لدينا: لكل متغير حالة

إشارة المرور // x\_sem; (في البداية 0) =  
int x\_count = 0;



يمكن تنفيذ عملية x.wait على النحو التالي:

```
x_count++;
إذا (العدد التالي (0) >
الإشارة (التالية)؛
آخر
إشارة (متزامنة)؛
انتظر (mes_x);
x_count--;
```





# مراقبة التنفيذ (تابع)



يمكن تنفيذ عملية `x.signal` على النحو التالي:

```
إذا (x_count > 0) { next_count++;
 إشارة (x_sem)؛ انتظر (التالي)؛
 next_count--;
}
```





# استئناف العمليات داخل الشاشة



إذا تم وضع عدة عمليات في قائمة الانتظار على الشرط  $x$ ، وتم تنفيذ  $x.signal()$ ، فأي منها يجب استئنافه؟



FCFS في كثير من الأحيان لا يكون كافيا



بناء الانتظار الشرطي من النموذج  $x.wait(c)$

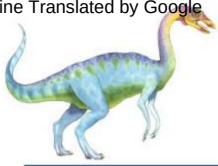


حيث  $c$  هو رقم الأولوية



تتم جدولة العملية ذات الرقم الأقل (أعلى أولوية) في المرة التالية





# تخصيص الموارد الفردية

- تخصيص مورد واحد بين العمليات المتنافسة باستخدام أرقام الأولوية التي تحدد الحد الأقصى للوقت الذي تخطط فيه العملية لاستخدام المورد

$R$ . اكتساب  $(t)$ ؛

...

الوصول إلى الموارد؛

...

$R$ . إطلاق سراح؛



حيث  $R$  هي مثال من نوع ResourceAllocator





# مراقب لتخصيص مورد واحد

مراقبة ResourceAllocator

{

بولياني مشغول؛ الشرط؛ x؛

void acquire(int time) {

إذا (مشغول)

tiaw.x(الوقت)؛

مشغول = صحيح؛

}

إصدار باطل { ()

مشغول x.signal();

= FALSE؛

}

كود التهيئة { ()

مشغول = خطأ؛

}

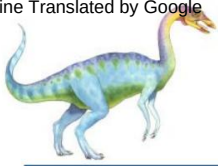




# أمثلة المزامنة

- سولاريس
- ويندوز
- لينكس
- خيوط P





# مزامنة سولاريس

- تنفيذ مجموعة متنوعة من الأقفال لدعم تعدد المهام، وتعدد العمليات (بما في ذلك العمليات في الوقت الفعلي)، وتعدد المعالجة
- يستخدم **المزامنة التكيفية** لتحقيق الكفاءة عند حماية البيانات من أجزاء التعليمات البرمجية القصيرة
- يبدأ كفل دوراني قياسي للإشارة الضوئية
- إذا تم قفل القفل، وبواسطة مؤشر ترابط يعمل على وحدة معالجة مركزية أخرى، يتم الدوران
- إذا تم الاحتفاظ بالقفل بواسطة مؤشر ترابط غير نشط، فسيتم حظره ووضع السكون في انتظار إشارة تحرير القفل
- يستخدم **متغيرات الحالة**
- يستخدم أقفال **القراء والكتاب** عندما تحتاج أقسام أطول من التعليمات البرمجية إلى الوصول إلى البيانات
- يستخدم **بوابات دوارة** لترتيب قائمة الخيوط التي تنتظر الحصول على قفل المزامنة التكيفية أو قفل القارئ والكتاب
- يتم تثبيت البوابات الدوارة لكل قفل على حدة، وليس لكل كائن
- يمنح وراثية الأولوية لكل بوابة دوارة الخيط الجاري تشغيله أعلى أولويات الخيط في بوابة الدوران الخاصة به

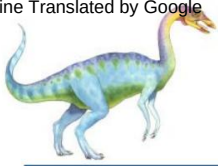




# مزامنة Windows

- يستخدم أقنعة المقاطعة لحماية الوصول إلى الموارد العالمية على أنظمة المعالج الأحادي
- يستخدم أقفال الدوران على أنظمة المعالجات المتعددة
  - لن يتم إيقاف قفل الخيط أبدًا
- يوفر أيضًا كائنات المرسل على أرض المستخدم والتي قد تعمل على تشغيل مفاتيح التبديل، وإشارات المرور، والأحداث، والمؤقتات
- الأحداث
  - يتصرف الحدث مثل متغير الشرط
- يقوم المؤقتون بإخطار موضوع واحد أو أكثر عند انتهاء الوقت
- كائنات المرسل إما في حالة الإشارة (الكائن متاح) أو في حالة عدم الإشارة (سيتم حظر الخيط)

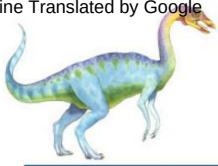




# مزامنة لينكس

- لينكس:
  - قبل إصدار kernel 2.6، تم تعطيل المقاطعات لتنفيذ الأقسام الحرجة القصيرة، الإصدار 2.6 والإصدارات الأحدث، وقائي بالكامل
  -
- يوفر لينكس:
  - إشارات المرور، الأعداد
  - الصحيحة الذرية ، أقفال
  - الدوران
  - إصدارات القارئ والكاتب لكلا منهما
- في نظام وحدة المعالجة المركزية الفردية، تم استبدال أقفال الدوران بتمكين وتعطيل استباق النواة





# مزامنة Pthreads

- واجهة برمجة تطبيقات Pthreads مستقلة عن نظام التشغيل
- ويوفر:
  - أقفال الميتكس
  - متغير الحالة
- تتضمن الامتدادات غير المحمولة ما يلي:
  - أقفال القراءة والكتابة
  - أقفال الدوران



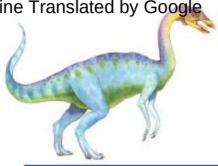


# النهج البديلة

---

- الذاكرة المعاملاتية
- أوبن إم بي
- لغات البرمجة الوظيفية





# الذاكرة المعاملاتية

- معاملة الذاكرة هي عبارة عن سلسلة من عمليات القراءة والكتابة إلى الذاكرة والتي يتم تنفيذها ذريًا.

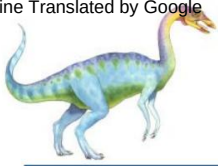
تحديث باطل ()

{

\*/قراءة/كتابة الذاكرة /\*

}





# أوبن إم بي

- OpenMP عبارة عن مجموعة من توجيهات المترجم وواجهة برمجة التطبيقات التي تدعم البرمجة المتوازية.

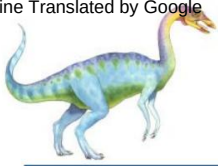
```
تحديث void (قيمة int)
{
 pragma omp #حرج {

 العد += القيمة

 }
}
```

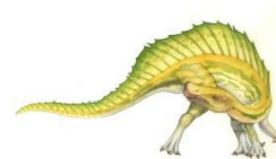
يتم التعامل مع الكود الموجود داخل التوجيه الحرج `pragma omp #باعتباره قسمًا حرجًا` ويتم تنفيذه ذريًا.





# لغات البرمجة الوظيفية

- تقدم لغات البرمجة الوظيفية نموذجًا مختلفًا عن اللغات الإجرائية حيث أنها لا تحافظ على الحالة.
- يتم التعامل مع المتغيرات على أنها غير قابلة للتغيير ولا يمكن تغيير حالتها بمجرد تعيين قيمة لها.
- هناك اهتمام متزايد باللغات الوظيفية مثل Scala وErlang لنهجها في التعامل مع سباقات البيانات.



# نهاية الفصل الخامس

